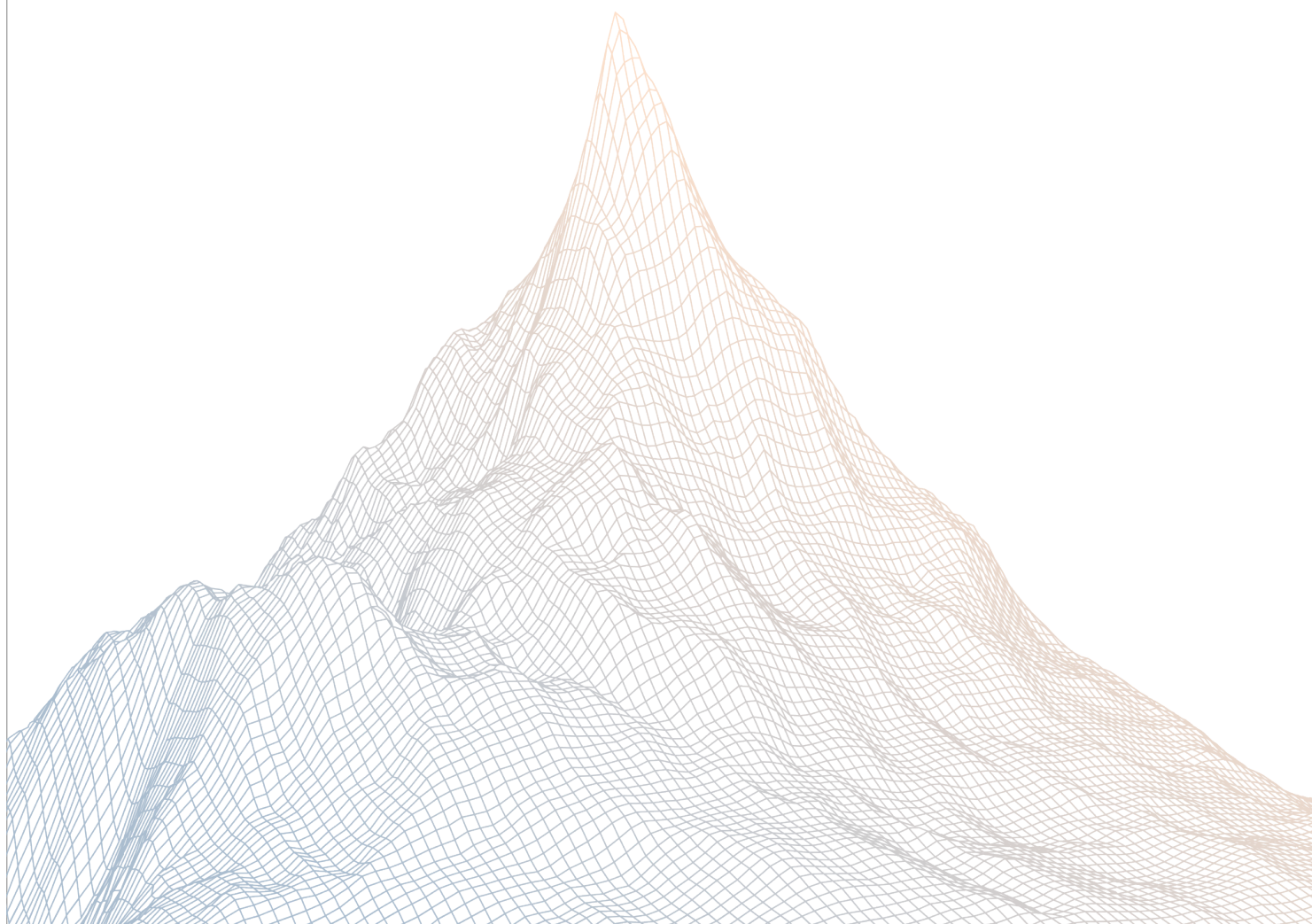


Meteora

Smart Contract Security Assessment

VERSION 1.1



Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Medium Risk	7
4.2	Low Risk	10

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	damm-v2
Repository	https://github.com/MeteoraAg/damm-v2/pull/177
Commit Hash	df501080ff4ef537b8abbf1aa8a6a45dd1625906
Files	Changes in PR-177

2.3 Audit Timeline

January 30, 2026	Audit start
February 2, 2026	Audit end
February 2, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	2
Informational	0
Total Issues	4

3

Findings Summary

ID	Description	Status
M-1	Mixed vesting will leave inner_vesting non-empty after completion, causing lock_inner_position to be blocked	Resolved
M-2	Missing external vesting validation for destination position in check_position_split_validity()	Resolved
L-1	fix_pool_fee_params fails to ensure new base-fee schedule is static	Resolved
L-2	Missing refresh_inner_vesting() before liquidity-lock operations can lock against stale vesting state	Resolved

4

Findings

4.1 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] Mixed vesting will leave inner_vesting non-empty after completion, causing lock_inner_position to be blocked

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [position.rs#L411-L414](#)
- [position.rs#L448-L451](#)

Description:

refresh_inner_vesting() is intended to finalize completed inner vesting and reset inner_vesting state when vesting has finished.

However, when external vesting exists, it means vested_liquidity > 0. That prevents refresh_inner_vesting() from resetting inner_vesting as it requires vested_liquidity = 0, leaving inner_vesting non-empty.

As a result, lock_inner_position will become unusable for the position because it requires position.inner_vesting.is_empty().

Recommendations:

Remove the self.vested_liquidity = 0 check in refresh_inner_vesting() and apply_split_inner_vesting() as follows

```
// redundant check self.vested_liquidity = 0 for safety
if self.inner_vesting.done()? && self.vested_liquidity = 0 {
  if self.inner_vesting.done()? {
    self.inner_vesting = InnerVesting::default();
  }
}
```

For `apply_split_inner_vesting()`, although external checks has ensured no external vestings, its better to be consistent with `refresh_inner_vesting()` and remove `self.vested_liquidity = 0` check..

Meteora: Resolved with [PR-184](#).

Zenith: Verified.

[M-2] Missing external vesting validation for destination position in `check_position_split_validity()`

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [ix_split_position2.rs#L105-L119](#)

Description:

`handle_split_position2()` will call `check_position_split_validity()` to validate that a position split is safe and preserves vesting/accounting invariants.

However, it fails to check whether the destination `second_position` has external vesting before allowing inner vesting to be split into it.

As a result, splitting inner vesting into a destination with external vesting can overwrite `second_position.vested_liquidity`, as `apply_split_inner_vesting()` will recalculate `vested_liquidity`.

This could cause existing `vested_liquidity` to be lost and unaccounted for.

Recommendations:

Update `check_position_split_validity()` to ensure that `second_position` do not have external vesting.

Meteora: Resolved with [PR-181](#).

Zenith: Verified.

4.2 Low Risk

A total of 2 low risk findings were identified.

[L-1] `fix_pool_fee_params` fails to ensure new base-fee schedule is static

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ix_fix_pool_fee_params.rs](#)

Description:

`fix_pool_fee_params` is intended to repair fee parameters without changing the effective fee policy.

However, they do not enforce `base_fee_handler_1.validate_base_fee_is_static()`, which allows the operator to extending the fee schedule window (e.g., increasing `number_of_period`, `period_frequency`, `max_limiter_duration`).

Recommendations:

In `fix_pool_fee_params`, require `base_fee_handler_1.validate_base_fee_is_static`.

Meteora: Resolved with [PR-183](#).

Zenith: Verified.

[L-2] Missing `refresh_inner_vesting()` before liquidity-lock operations can lock against stale vesting state

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- `lock_position`
- `lock_inner_position`
- `permanent_lock_position`

Description:

The `remove_liquidity()` IX will call `position.refresh_inner_vesting()` to update `position.unlocked_liquidity` and `position.vested_liquidity` to ensure vesting state is current before any liquidity-modifying operation.

However, the following IXs fail to call `position.refresh_inner_vesting()` to update the vesting state before performing its operations.

- `lock_position` calls `position.lock(...)` without refreshing
- `lock_inner_position` checks `inner_vesting.is_empty()` on stale state,
- `permanent_lock_position` calls `permanent_lock_liquidity(...)` without refreshing.

This makes them inconsistent with `remove_liquidity()` and require a separate `refresh_vesting` IX call beforehand.

Recommendations:

Call `position.refresh_inner_vesting()` at the start of `lock_position`, `lock_inner_position`, and `permanent_lock_position` before any state checks or liquidity-locking logic.

Meteora: Resolved with [PR-185](#).

Zenith: Verified.